CHAPTER 8

Regular Expressions

We spend much of our time in Ruby working with strings, so it seems reasonable for Ruby to have great tools for working with those strings. As we've seen, the String class itself is no slouch—it has more than 100 methods. But there are still things that the basic String class can't do on its own. For example, we might want to see whether a string contains two or more repeated characters, or we might want to replace every word longer than fifteen characters with its first five characters and an ellipsis. This is when we turn to the power of regular expressions.

Regular expressions are powerful and are used in many languages besides Ruby. In this chapter, we'll cover the basics of what regular expressions can do in Ruby; later, in Regular Expressions, on page 441, we'll show you all the details and some more complex techniques.

What Regular Expressions Let You Do

A regular expression is a pattern that can be matched against a string. It can be a simple pattern, such as *the string must contain the sequence of letters "cat,"* or the pattern can be complex, such as *the string must start with a protocol identifier, followed by two literal forward slashes, followed by…*, and so on. This is cool in theory. But what makes regular expressions so powerful is what you can do with them in practice:

- You can test a string to see whether it matches a pattern.
- You can extract from a string the sections that match all or part of a pattern.
- You can change the string, replacing parts that match a pattern.

Ruby provides built-in support that makes pattern matching, extraction, and substitution convenient and concise. In this first section, we'll work through the basics of regular expression patterns and see how Ruby supports matching and replacing based on those patterns. In the sections that follow, we'll dig deeper into both the patterns and Ruby's support for them.

Creating and Using Regular Expressions

There are many ways of creating a regular expression pattern in Ruby. By far the most common is to write the pattern between forward slashes. Thus, the pattern /cat/ is a regular expression literal in the same way that "cat" is a string literal.